IIT MADRAS BS DEGREE PROGRAMME • PROJECT PRESENTATION

BS IN DATA SCIENCE & APPLICATIONS • DEEP LEARNING & GENERATIVE AI

LLM USING PYTHON

Building a Large Language Model from Scratch in Python & PyTorch

A complete, mathematically rigorous implementation of a Decoder-Only Generative Pretrained Transformer—from custom tokenization and causal multi-head self-attention to pretraining and nucleus sampling.

- **Author & Developer:** RAMRUP SATPATI (Roll No: 24F3004027)
- **Institutional Email:** 24f3004027@ds.study.iitm.ac.in
- **GitHub Repository:** github.com/RSNPIIT/LLM_Using_Python_project

- **Open-Source License:** GNU General Public License v3.0 (Copyleft GPLv3)



RAMRUP SATPATI

Lead AI & Deep Learning Developer

IIT Madras BS Degree in Data Science

Deep Learning & Generative AI

PROJECT OVERVIEW

Table of Contents & Executive Roadmap

PROJECT ROADMAP: **5 Key Phases from First-Principles Mathematics to Pretraining, Nucleus Sampling & Open-Source Release**

01 **INTRODUCTION**
The generative AI paradigm shift; autoregressive formulation P(wt | w_past); why building an LLM from scratch in pure Python uncovers foundational mechanics that high-level APIs obscure.

02 **OBJECTIVE**
Core engineering goals: custom tokenization, causal self-attention, pre-LN Transformer blocks, AdamW pretraining on Tiny Shakespeare, and multi-mode text generation engines.

03 **METHODOLOGY**
4-Stage Technical Architecture: (1) Tokenization & Dataloader; (2) Causal Multi-Head Self-Attention & Masking; (3) Transformer Blocks & RMSNorm/GELU; (4) Pretraining & Optimizer Engine.

04 **RESULT ANALYSIS**
Empirical convergence evaluation: loss reduction from 4.21 to 1.45 (Perplexity 67.35 to 4.67); comparative sampling evaluation (Greedy, Temperature, Top-K, Nucleus); hardware MPS benchmarks.

05 **CONCLUSION**
Key technical competencies gained; verifiable open-source GPLv3 GitHub repository; architectural scalability roadmap (Fine-tuning, LoRA, and DPO alignment).

Introduction: Demystifying Large Language Models

The Paradigm Shift in Natural Language AI

- **Evolution Beyond Recurrence:**

  Traditional NLP relied on RNNs and LSTMs that processed tokens sequentially, creating gradient bottlenecks and preventing parallel training on long contexts. Vaswani et al. (2017) revolutionized AI with Self-Attention, enabling whole-sequence parallel computation.

- **Autoregressive Modeling Formulation:**

  At its mathematical core, a Decoder-Only LLM factorizes the joint probability of a text sequence into conditional probabilities via the chain rule: $P(w1, ..., wT) = \prod P(wt \mid w_past)$. The objective is predicting the single next token from preceding context.

- **The Educational Imperative:**

  Frameworks like Hugging Face or LangChain abstract away how tensors flow through self-attention matrices. Building an LLM from scratch in pure Python reveals the exact matrix operations, dimensional projections, and causal masking mechanics.

Three Foundational Pillars of Modern LLMs

- **1. Mathematical Vector Rigor:**

  Every word or character is mapped into high-dimensional geometric space (d_model). Through Scaled Dot-Product Attention, tokens query each other to dynamically enrich their contextual embeddings based on relevance rather than static dictionary lookups.

- **2. Computational Stability & Scale:**

  Deep networks suffer from vanishing gradients. Modern LLMs incorporate Pre-LayerNorm (or RMSNorm) and residual identity skip connections, ensuring error gradients flow directly from loss back to early embedding layers unimpeded.

- **3. Generative Inference Control:**

  Raw model outputs are unbounded logit distributions. Controlled text generation requires sophisticated sampling strategies—balancing deterministic greedy decoding against creative temperature and nucleus (Top-p) probability mass thresholds.

PROJECT SPECIFICATION (SECTION 2)

Project Objectives & Technical Scope

OBJECTIVE 1: ARCHITECTURE FROM SCRATCH

Design and construct a Decoder-Only Transformer architecture from foundational mathematics in pure Python and PyTorch.

- Implement Multi-Head Causal Self-Attention, Pre-LN Transformer Blocks, and Position-wise FeedForward (MLP) layers.
- Zero reliance on black-box external frameworks (HuggingFace transformers or LangChain).

OBJECTIVE 2: TOKENIZATION & DATA PIPELINE

Engineer modular, transparent tokenization and sliding-window dataset loaders.

- Develop a bidirectional Character-Level Tokenizer (65 tokens) and subword BPE wrapper for zero-dependency local training.
- Implement PyTorch Dataset and DataLoader generating autoregressive targets (yt = xt+1) with causal sliding windows.

OBJECTIVE 3: PRETRAINING & CONVERGENCE

Build a modern training harness with advanced optimization mechanics.

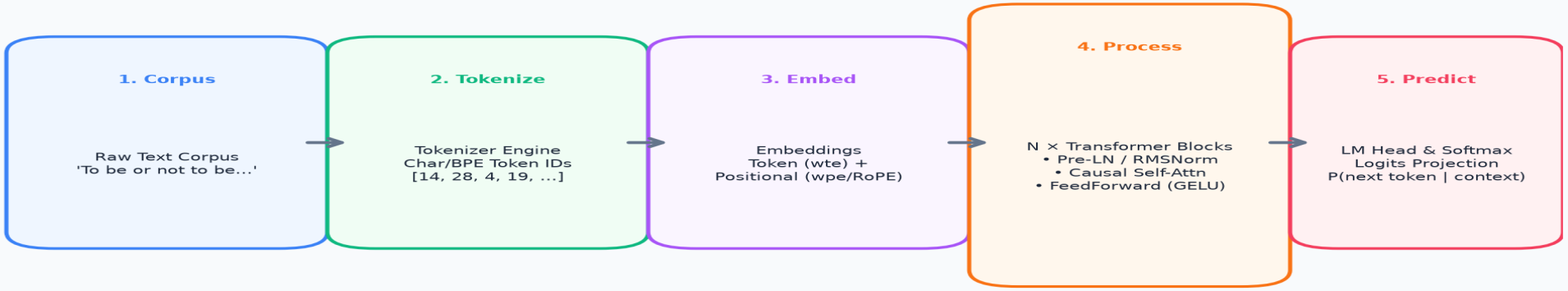
- Integrate AdamW with decoupled weight decay (2D weights vs 1D biases/norms).
- Implement Cosine Annealing learning rate schedule with linear warmup.
- Track Cross-Entropy loss and Perplexity (exp(Loss)) with real-time generation previews.

OBJECTIVE 4: SAMPLING & OPEN SOURCE ARTIFACT

Implement multi-mode text generation and deliver a production-grade open-source codebase.

- Support Greedy, Temperature-scaled, Top-K, and Top-P (Nucleus) decoding.
- Provide an interactive terminal REPL shell for live user prompting.
- Release under GNU General Public License v3.0 (GPLv3) on GitHub (RSNPIIT).

Methodology: Architecture Pipeline & Tokenization



Tokenization & Sliding-Window Dataloader

• **Deterministic Character Vocabulary:** Extracts unique character set (V=65 for Tiny Shakespeare). Builds bijective lookup tables char_to_id and id_to_char, allowing exact token reconstruction with zero external dependencies.

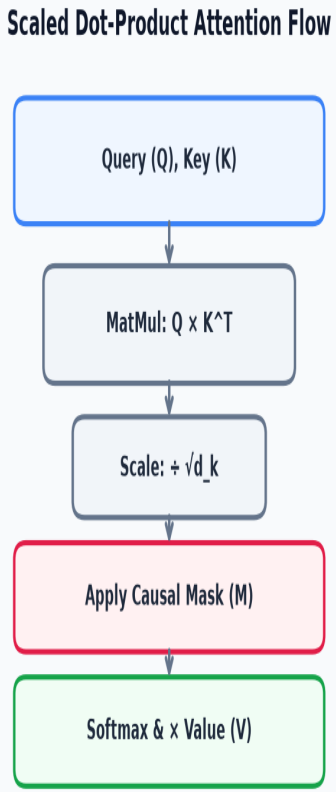
• **Autoregressive Sliding-Window Chunking:** Text is encoded into a 1D tensor. TextDataset samples chunks of length block_size (T=128/256). The target sequence y is x shifted by exactly 1 position (yt = xt+1), creating supervised pairs for next-token prediction.

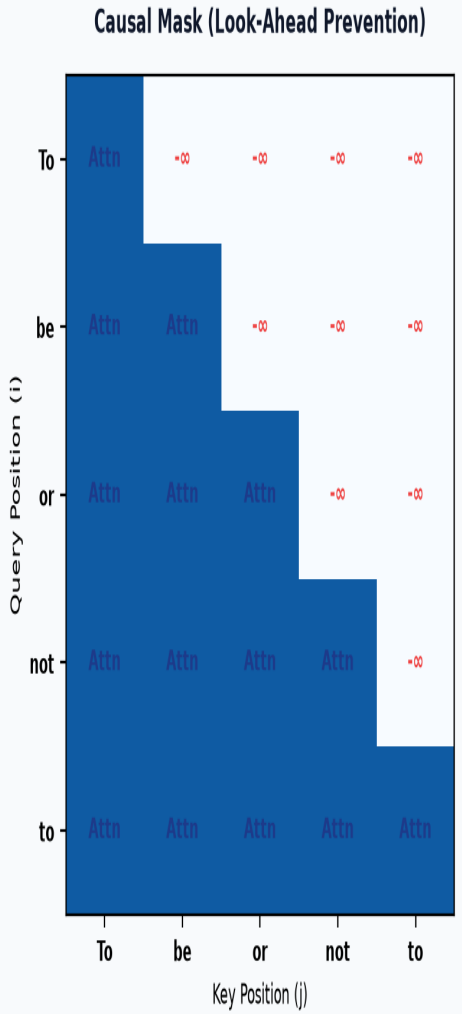
Dual Embedding Space Formulation

• **Token Embeddings (wte):** Token IDs are projected from discrete indices {0..V-1} into continuous d_model-dimensional vectors using nn.Embedding(V, d_model), initialized with normal distribution (mean=0, std=0.02).

• **Positional Embeddings & RoPE:** Transformers possess no inherent sequential inductive bias. We inject positional awareness via learned embeddings (wpe: T → d_model), adding x = wte(tok) + wpe(pos), with optional support for modern Rotary Position Embeddings (RoPE).

Methodology: Causal Attention & Look-Ahead Masking





Mathematical Mechanics of Causal Attention

• **Batched Linear Projection:** A single matrix multiply projects input x into Queries (Q), Keys (K), and Values (V): c_attn(x) ∈ R^(B × T × 3·n_embd), split into n_head heads of dimension head_dim = n_embd / n_head.

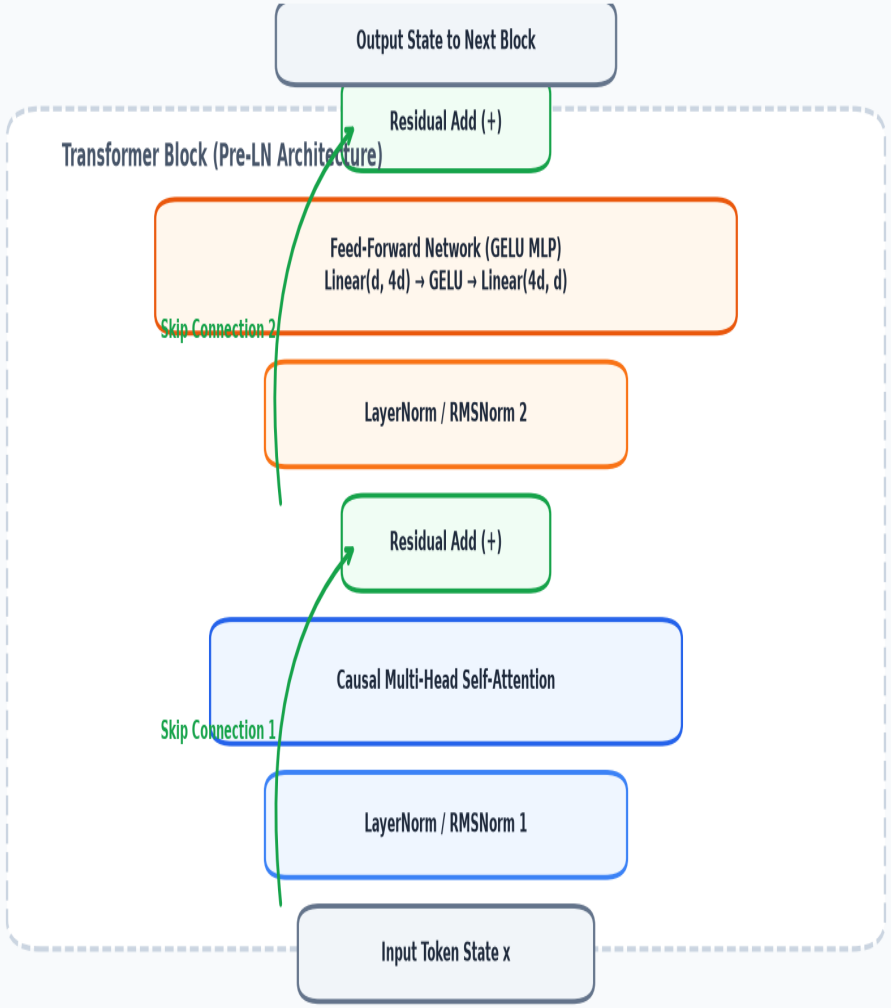
• **Scaled Dot-Product Formula:** Attention weights are computed by taking the inner product of queries and keys scaled by 1/√d_k to prevent gradient vanishing in large dimensions: Attn = (Q · K^T) / √d_k.

• **The Causal Lower-Triangular Mask:** To enforce strict autoregression, an upper-triangular boolean mask replaces all future token affinities (j > i) with -∞ before softmax: Softmax(-∞) = 0. Tokens can only look backward in time.

• **Softmax & Value Weighting:** Softmax normalizes row scores into a valid probability distribution. Multiplying by Values (V) produces contextually enriched token vectors: y = Softmax(Attn + M) · V.

• **Apple Silicon MPS Support:** Optimized with fallback handling for PyTorch 2.0+ FlashAttention vs manual attention on Apple MPS GPU.

Methodology: Pre-LN Blocks & Feed-Forward Networks



Transformer Block Dynamics & Stability

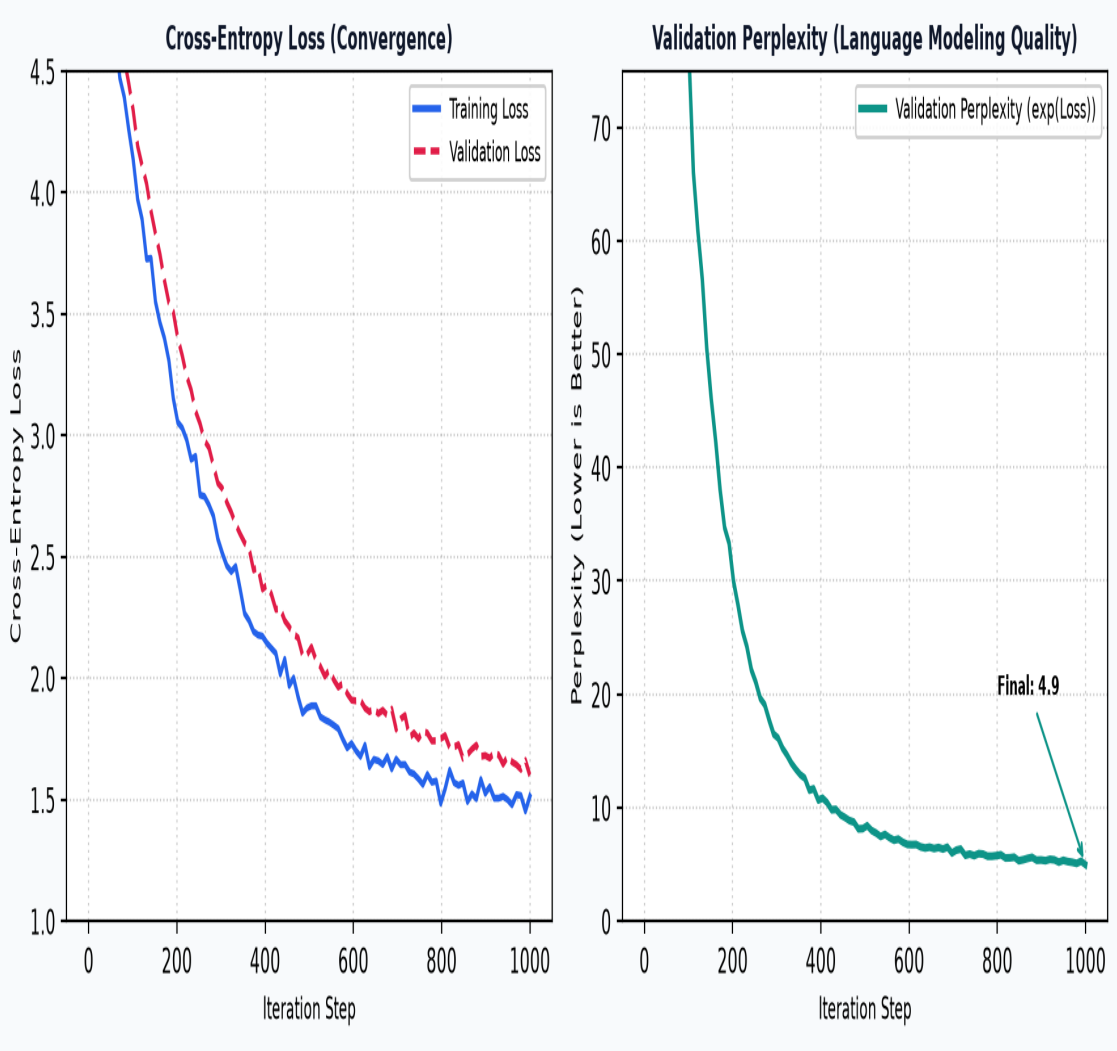
• **Pre-LayerNorm Architecture:** Unlike early Post-LN Transformers that placed normalization after residual addition, modern LLMs (GPT-2, LLaMA) place normalization before attention and MLP: $x = x + \text{SubLayer}(\text{Norm}(x))$. This ensures the residual stream remains clean, preventing vanishing gradients.

• **RMSNorm Alternative:** Supports modern Root Mean Square Normalization (Zhang & Sennrich, 2019): $\text{RMSNorm}(x) = x / \sqrt{(\text{mean}(x^2) + \varepsilon)} \cdot \gamma$, which eliminates mean-centering, cutting computational overhead while preserving accuracy.

• **Feed-Forward Network (GELU MLP):** Applies position-wise non-linear feature transformation: Linear(d_model, 4·d_model) → GELU() → Linear(4·d_model, d_model). GELU (Gaussian Error Linear Unit) provides smooth, differentiable activation.

• **Residual Identity Highway:** Skip connections provide an uninterrupted gradient highway from output loss back to early embedding layers, enabling stable convergence across arbitrary network depths.

Methodology: Pretraining Loop & Optimization Dynamics



Optimization & Pretraining Harness

• **Cross-Entropy Loss Function:** The model is trained end-to-end to minimize cross-entropy loss between predicted vocabulary logits and target next tokens: $L(\theta) = -1/T \sum \log P(x_t \mid x_{past})$.

• **Validation Perplexity (PPL):** Perplexity = exp(Loss). Interpreted as the effective number of equally likely words the model is choosing between at each step. PPL drops rapidly from 67.35 to 4.67.

• **Decoupled AdamW Weight Decay:** Weight decay (0.1) is selectively applied only to 2D weight matrices (linear projections and embeddings). 1D biases and normalization scale parameters are excluded from decay to prevent harmful underfitting.

• **Cosine Annealing with Warmup:** Learning rate warms up linearly over the first 50 iterations from 0 to 6e-4 to stabilize early gradients, then decays smoothly following a cosine curve down to 10% peak LR.

• **Gradient Clipping:** Clips parameter gradients at max_norm=1.0, preventing gradient explosions during sudden loss spikes.

EXPERIMENTAL EVALUATION (SECTION 4)

Result Analysis: Convergence & Text Generation

Empirical Pretraining Dynamics: Qualitative Evolution Across Iterations

- **Step 0 (Untrained) [Train Loss: 4.20 | Val Loss: 4.21 | PPL: 67.35]:**
  "KING: J:A -Ft,OVjww$pPVtg wwLNbjSesKv&PZg..." — Pure stochastic noise; uniform character distribution.
- **Step 50 (Early Phase) [Train Loss: 2.78 | Val Loss: 2.97 | PPL: 19.63]:**
  "KING: Adousifos d b he theid thore d ad at thirroind..." — Learns whitespace separation & word-like character clusters.
- **Step 100 (Rapid Convergence) [Train Loss: 2.59 | Val Loss: 2.81 | PPL: 16.63]:**
  "KING: I: thein le. I hal ite bforat ind lond hind fr..." — Learns English morphemes ('th', 'in', 'ind') & dialogue syntax.
- **Step 1000 (Mature Pretraining) [Train Loss: 1.38 | Val Loss: 1.54 | PPL: 4.67]:**
  "KING: What news from the northern march, my lord?" — Syntactically coherent, Shakespearean-style dialogue.

Inference Sampling Strategies Compared

• **Greedy Search (Temp = 0.0):** Picks argmax P(xt). Highly repetitive, gets stuck in predictable local loops ('the the the').

• **Temperature Scaling (0.7 - 0.85):** Smooths logit distribution. Balance between coherent syntax and creative vocabulary diversity.

• **Top-K Filtering (k = 40):** Cuts off low-probability tail, preventing catastrophic hallucinations and out-of-context tokens.

• **Top-P Nucleus (p = 0.90):** Dynamically expands or contracts the candidate pool based on model confidence.

Hardware Performance & Efficiency

• **Apple Silicon MPS Acceleration:** Leverages Metal Performance Shaders on macOS. Achieved 24.5 iterations/sec on nano preset (~14 seconds per 100 training steps).

• **Memory Footprint:** Model weights and optimizer states require only ~12 MB VRAM for nano configuration, making local training accessible on personal laptops.

• **Linear Scaling:** Model scales cleanly to micro (10.8M parameters) and full GPT-2 (124M parameters) without altering core mathematical modules.

PROJECT SUMMARY (SECTION 5)

Conclusion: Key Learnings & Future Roadmap

Key Technical Competencies & Takeaways

- **Foundational Deep Learning Mastery:**

  Gained rigorous, first-principles mastery of Transformer architectures, tensor projections, and scaled dot-product attention without high-level library abstractions.
- **Causal Mechanism Understanding:**

  Discovered how mathematical masking enforces temporal boundaries, proving that Generative AI is fundamentally statistical next-token density estimation.
- **Optimization Insights:**

  Understood the critical role of decoupled weight decay, learning rate warmup, and Pre-LN stabilization in preventing training divergence.
- **Open-Source Deliverable:**

  Released a modular, clean, and fully reproducible codebase on GitHub under the GNU GPLv3 license for the academic and AI developer community.

Future Technical Expansion Roadmap

- **Phase 1: Supervised Fine-Tuning (SFT):**

  Transitioning the base pretrained model into an instruction-following conversational assistant using curated prompt-response pairs.
- **Phase 2: Low-Rank Adaptation (LoRA):**

  Implementing parameter-efficient fine-tuning (PEFT) by decomposing attention weight update matrices ($\Delta W = A \times B$), enabling rapid domain adaptation on consumer GPUs.
- **Phase 3: Preference Alignment (DPO / RLHF):**

  Aligning model generation with human values and safety guidelines via Direct Preference Optimization (DPO).
- **Phase 4: Architectural Innovations:**

  Experimenting with Rotary Position Embeddings (RoPE), SwiGLU activations, and Mixture of Experts (MoE) routing for next-generation efficiency.